College0

Phase II: Design Report

Course	CSC322 Software Engineering
Team	Team Y
Authors	Atai Kydyrov, Abdullah Zidan, Syed Mahmood, Justin Jones
Version	1.0
Date	April 23, 2026

Table of Contents

1. Introduction & Collaboration Class Diagram
2. Use Cases — Scenarios & Diagrams
3. Entity-Relationship Diagram
4. Detailed Design — Pseudo-code
5. System Screens & Prototype
6. Group Meeting Memos
7. Git Repository

1. Introduction & Collaboration Class Diagram

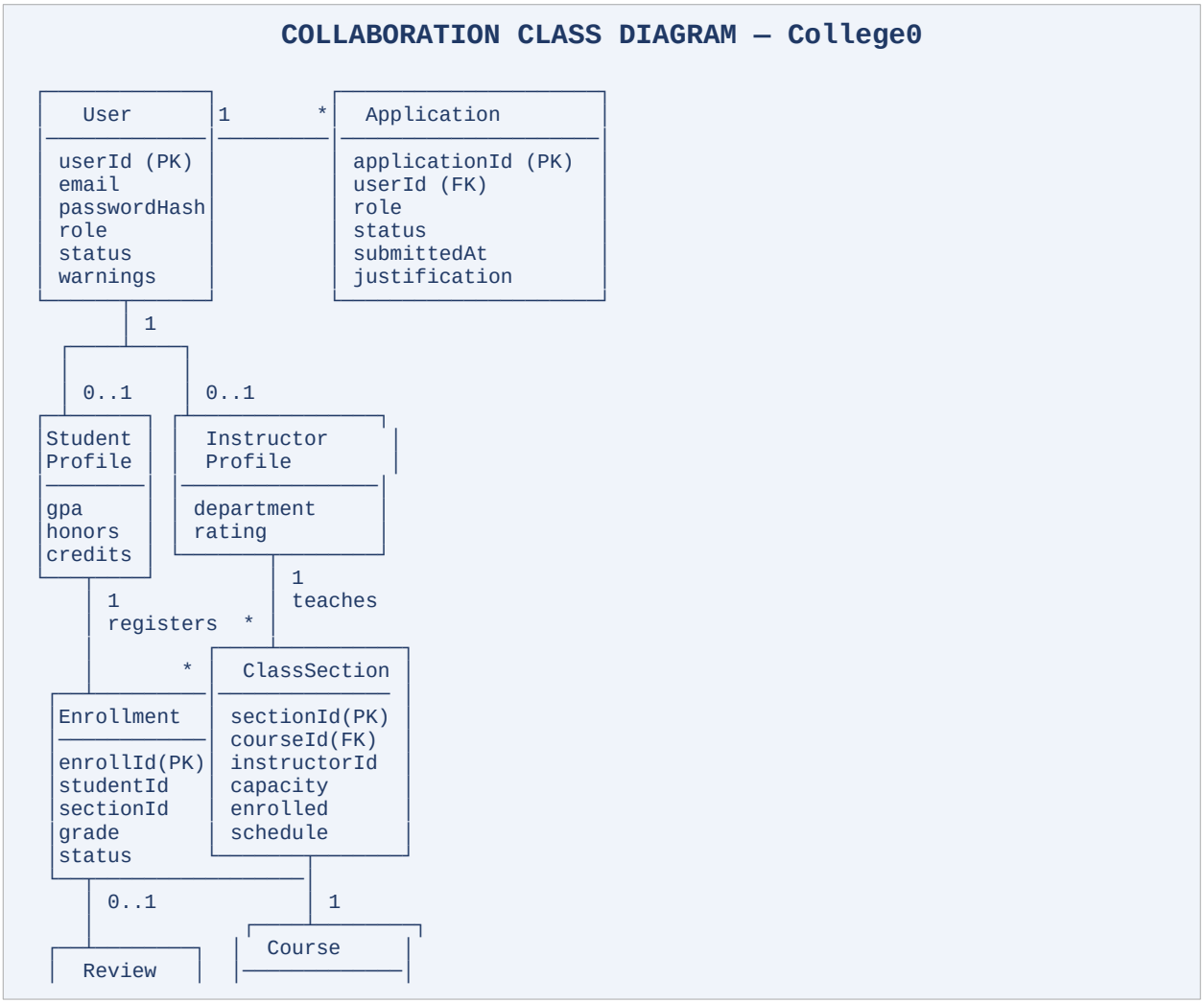
College0 is an AI-enabled college management system that simulates realistic educational administration workflows. It supports four actor roles — Visitor, Student, Instructor, and Registrar — across a semester lifecycle with four phases: Setup, Registration, Running, and Grading. The system enforces complex academic policies, provides waitlist management, review/rating workflows with taboo-word moderation, GPA-driven disciplinary actions, complaint handling, and an AI Knowledge Assistant with vector-DB retrieval and LLM fallback.

The architecture is organized into five subsystems:

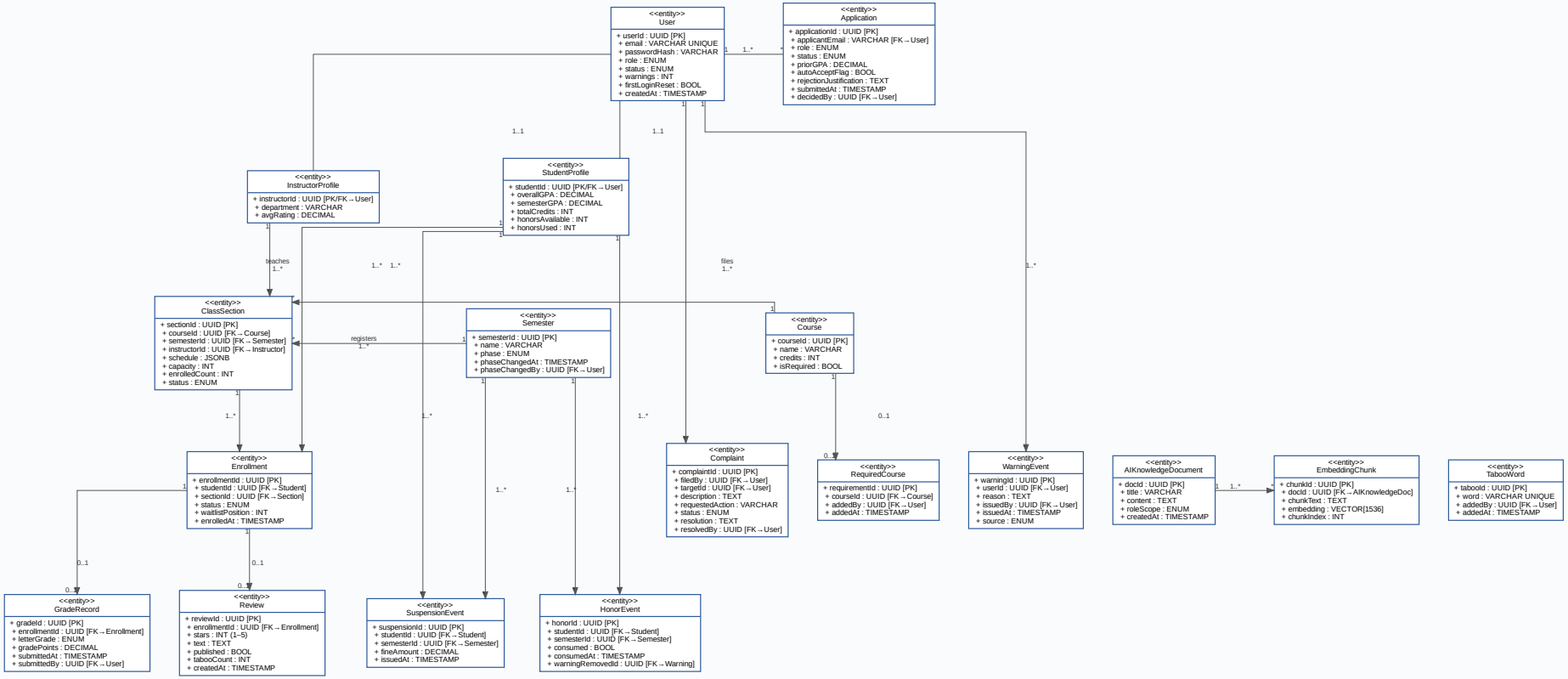
- Identity & Access — authentication, RBAC, password management
- Academic Operations — semester phases, courses, enrollment, grading
- Conduct & Governance — reviews, moderation, complaints, warnings, suspensions
- AI Assistant — RAG retrieval with LLM fallback and hallucination warnings
- Analytics — public dashboard, honor roll, graduation

1.1 Collaboration Class Diagram (System Overview)

The diagram below shows the major classes, their key attributes, and the relationships that drive the system. Multiplicity labels indicate cardinality between entities.



College0 — Collaboration Class Diagram



2. All Use Cases — Scenarios & Diagrams

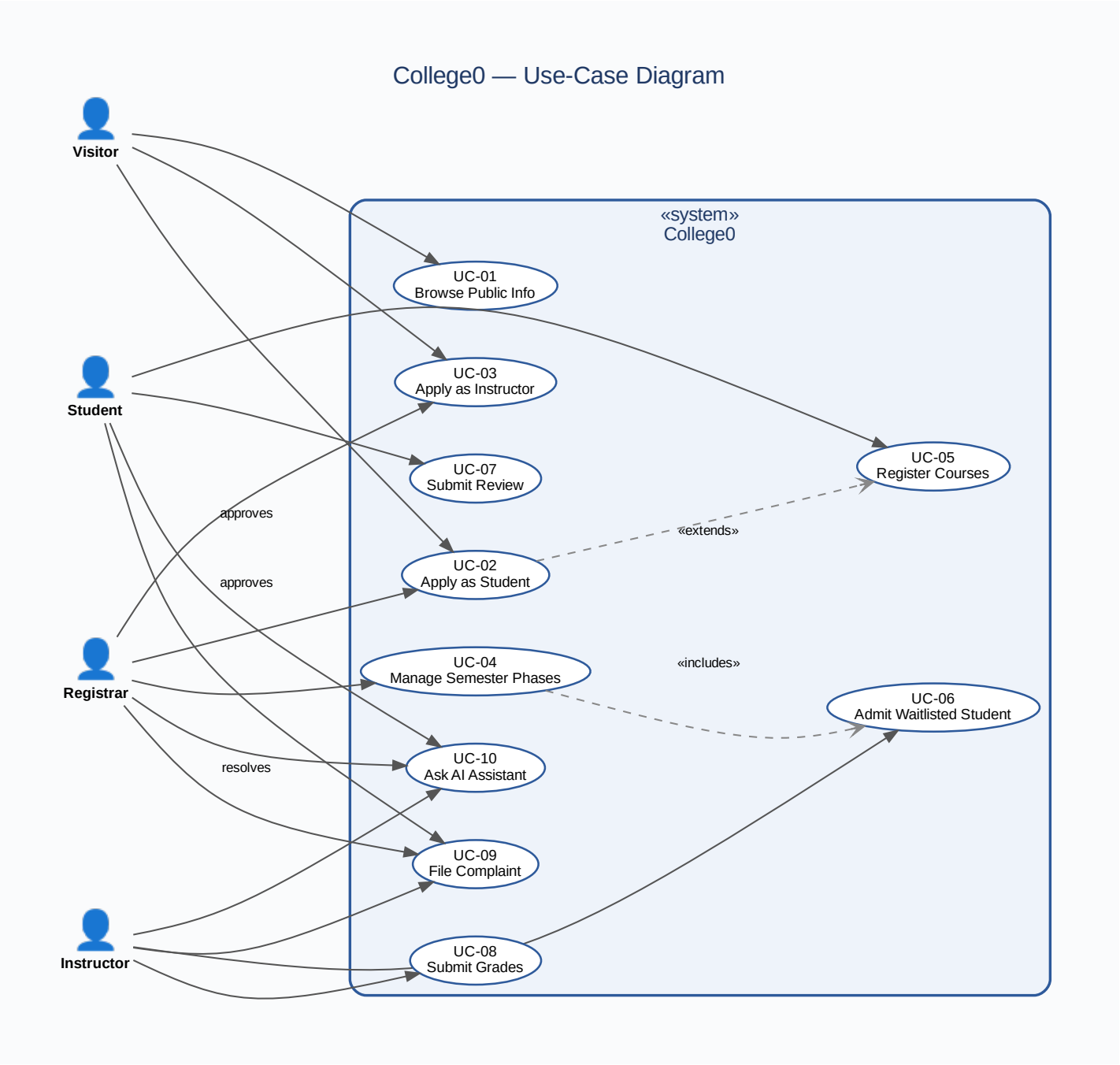
This section covers all ten major use-case groups. For each, we provide: actors, preconditions, normal scenario, exceptional scenarios, and a collaboration/sequence diagram. Three use cases (UC-04, UC-07, UC-08) are additionally represented as Petri nets.

2.0 Use-Case Diagram (Actor–Use-Case Overview)

Use Case ID	Use Case Name	Primary Actor(s)	Secondary Actor(s)
UC-01	Browse Public Info	Visitor	—
UC-02	Apply as Student	Visitor	Registrar
UC-03	Apply as Instructor	Visitor	Registrar
UC-04	Manage Semester Phases	Registrar	System
UC-05	Register Courses	Student	System
UC-06	Admit Waitlisted Student	Instructor	Student, System
UC-07	Submit Review	Student	System, Registrar
UC-08	Submit Grades	Instructor	System, Registrar
UC-09	File Complaint	Student / Instructor	Registrar
UC-10	Ask AI Assistant	Any User	AI Subsystem

USE-CASE DIAGRAM — College0

```
Visitor      <<System>> College0
             (UC-01) Browse Public Info
             (UC-02) Apply as Student
             (UC-03) Apply as Instructor

Student      (UC-05) Register Courses
             (UC-07) Submit Review
             (UC-09) File Complaint (as filer)
             (UC-10) Ask AI Assistant

Instructor   (UC-06) Admit Waitlisted Student
             (UC-08) Submit Grades
             (UC-09) File Complaint (as filer)
             (UC-10) Ask AI Assistant

Registrar    (UC-02) Approve/Reject Student App
             (UC-03) Approve/Reject Instructor App
             (UC-04) Manage Semester Phases
             (UC-09) Resolve Complaints
             (UC-10) Ask AI Assistant
```


College0 — Use-Case Diagram
Visitor
Student
Registrar
Instructor
«system»
College0
UC-01
Browse Public Info
UC-03
Apply as Instructor
UC-07
Submit Review
UC-02
Apply as Student
UC-04
Manage Semester Phases
UC-10
Ask AI Assistant
UC-09
File Complaint
UC-08
Submit Grades
UC-05
Register Courses
UC-06
Admit Waitlisted Student
approves
approves
resolves
«extends»
«includes»

UC-01: Browse Public Info

Field	Details
Actor	Visitor (unauthenticated)
Precondition	None — system is publicly accessible
Postcondition	Visitor views landing page; no state changes
Trigger	Visitor navigates to the College0 URL

Normal Scenario

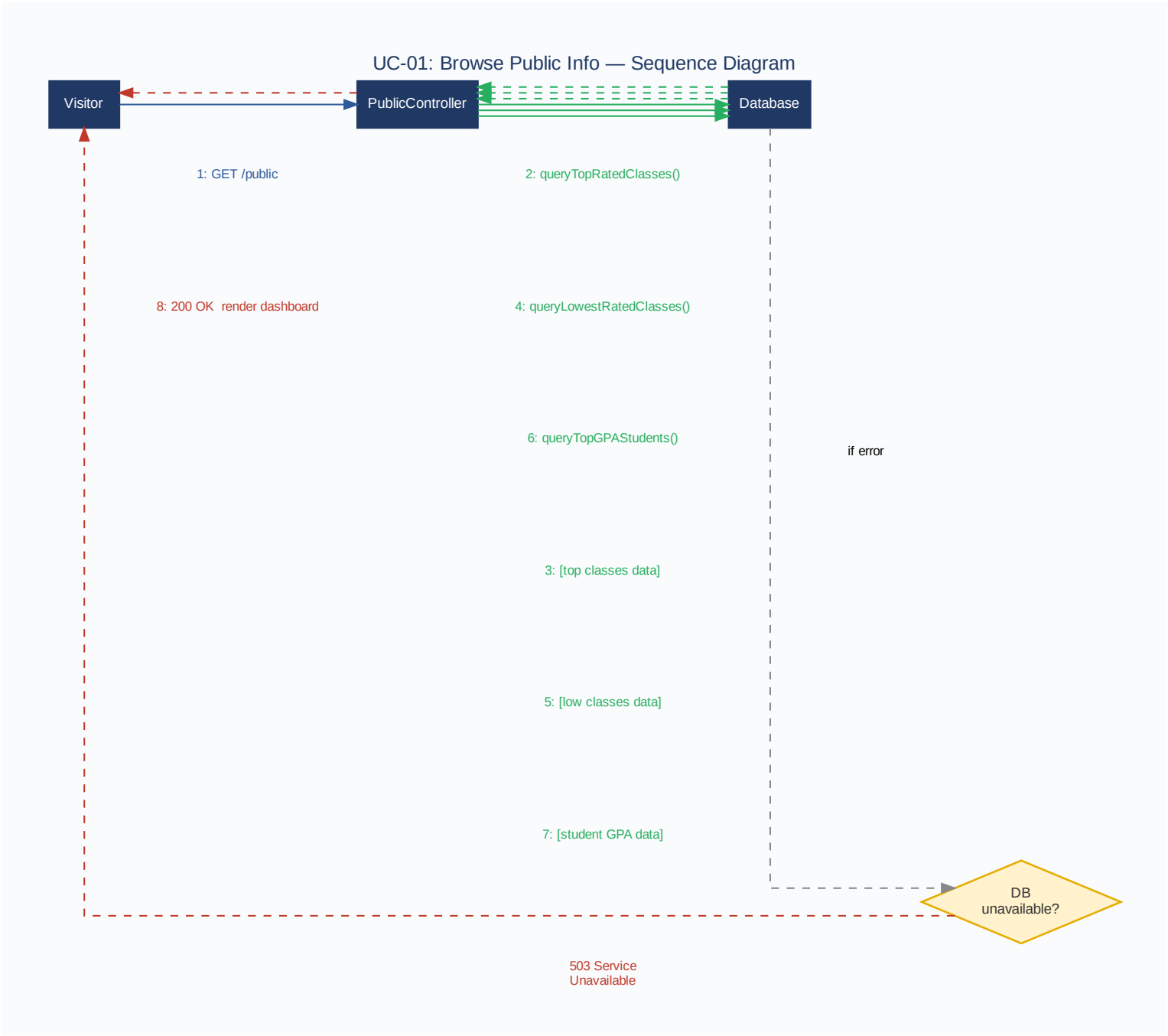
1. Visitor opens browser and navigates to College0 root URL.
2. System renders public dashboard: college description, top-rated classes (sorted desc by avg rating), lowest-rated classes, and top-GPA students.
3. Visitor reads information; no login required.

Exceptional Scenarios

- Database unavailable: System shows a 503 error page with a retry message.
- No classes rated yet: Sections display 'No data yet' placeholders.
- No students enrolled: GPA leaderboard shows empty state.

Collaboration Diagram — UC-01

```
Visitor                PublicController          DB
  |                          |                     |
  |──1: GET /public──────────>|                     |
  |                          |──2: queryTopClasses──>|
  |                          <──────────────────────-|
  |                          |──3: queryLowClasses──>|
  |                          <──────────────────────|
  |                          |──4: queryTopGPA──────>|
  |                          <──────────────────────|
  |<──5: render dashboard────|                     |
```


UC-01: Browse Public Info — Sequence Diagram
Visitor
PublicController
Database
1: GET /public
2: queryTopRatedClasses()
8: 200 OK render dashboard
4: queryLowestRatedClasses()
6: queryTopGPAStudents()
if error
3: [top classes data]
5: [low classes data]
7: [student GPA data]
DB unavailable?
503 Service Unavailable

UC-02: Apply as Student

Field	Details
Actor	Visitor (applicant), Registrar (reviewer)
Precondition	System is in any phase; applicant has not previously applied
Postcondition	Application created; if approved: Student account created with temp password
Trigger	Visitor clicks 'Apply as Student' and submits form

Normal Scenario

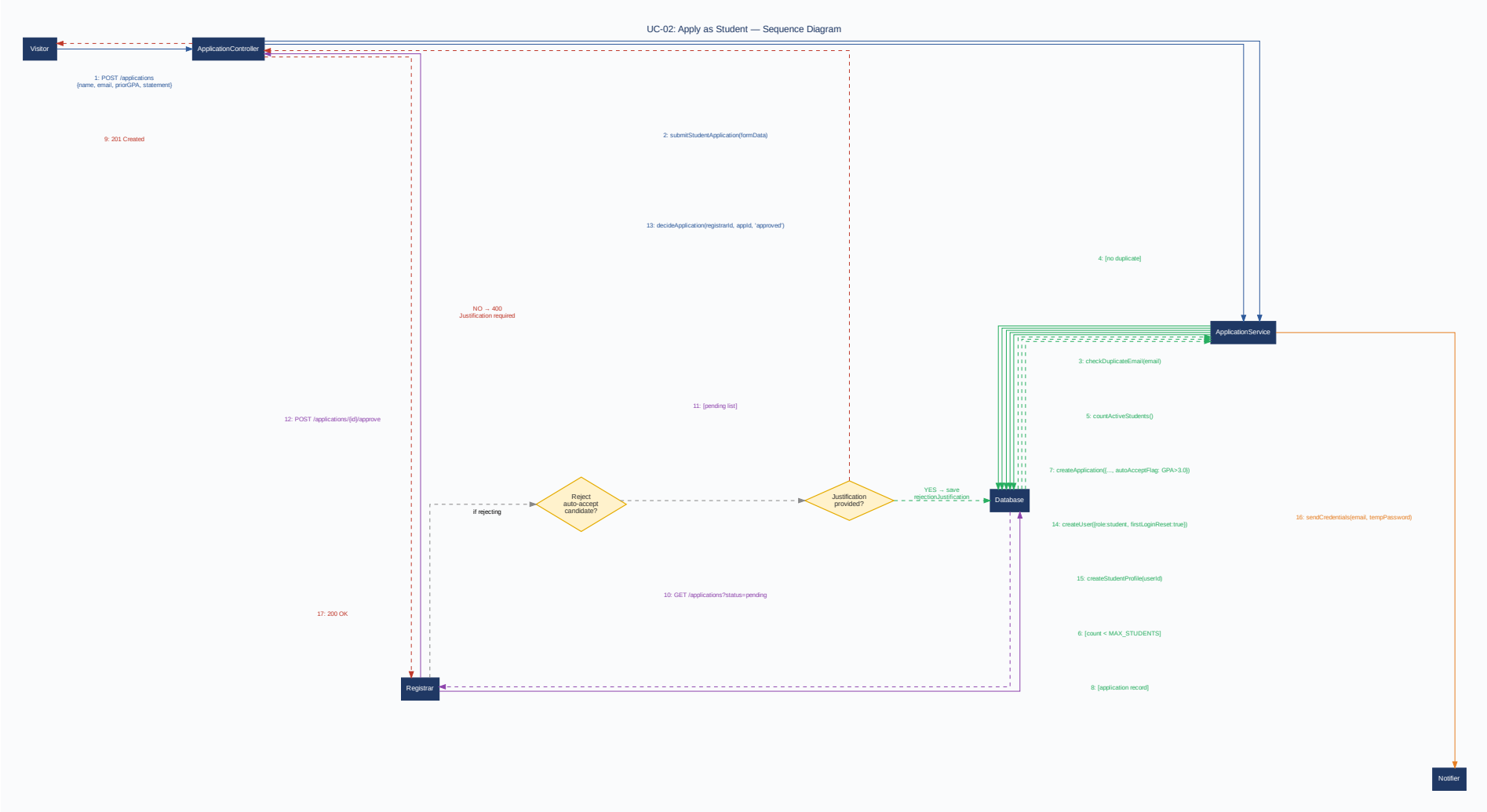
4. Visitor fills in application form (name, email, prior GPA, personal statement) and submits.
5. System checks if prior GPA > 3.0 and student quota not reached; sets auto-accept flag.
6. Registrar opens pending applications list and reviews.
7. Registrar approves application; system creates Student account with unique ID and temporary password; first-login reset flag set.
8. System emails credentials (in demo: displays on screen).

Exceptional Scenarios

- Registrar rejects auto-accept eligible applicant: Must enter textual justification stored in Application record; applicant can view it.
- Duplicate email: System returns validation error before submission.
- Quota reached: Auto-accept flag set to false regardless of GPA.
- Form incomplete: Client and server validation reject with field-level messages.

Collaboration Diagram — UC-02

```
Visitor        AppController      DB          Registrar
   |                |               |               |
   |--1:POST /app-->|               |               |
   |                |--2:checkGPA-->|               |
   |                <---------------|               |
   |                |--3:saveApp--->|               |
   <--4:201 OK------|               |               |
   |                |               |<-5:listApps---|
   |                |               |--6:appData--->|
   |                <-------------------7:approve---|
   |                |--8:createUser>|               |
   |                |--9:sendCreds->Visitor         |
```


UC-02: Apply as Student — Sequence Diagram
Visitor
ApplicationController
ApplicationService
Database
Registrar
Notifier
1: POST /applications
{name, email, priorGPA, statement}
2: submitStudentApplication(formData)
3: checkDuplicateEmail(email)
4: [no duplicate]
5: countActiveStudents()
6: [count < MAX_STUDENTS]
7: createApplication({..., autoAcceptFlag: GPA>3.0})
8: [application record]
9: 201 Created
10: GET /applications?status=pending
11: [pending list]
12: POST /applications/{id}/approve
13: decideApplication(registrarId, appId, 'approved')
14: createUser({role:student, firstLoginReset:true})
15: createStudentProfile(userId)
16: sendCredentials(email, tempPassword)
17: 200 OK
Justification provided?
YES → save rejectionJustification
NO → 400
Justification required
Reject auto-accept candidate?
if rejecting

UC-03: Apply as Instructor

Field	Details
Actor	Visitor (applicant), Registrar (reviewer)
Precondition	No existing account with same email
Postcondition	Instructor account created; assignable to classes
Trigger	Visitor clicks 'Apply as Instructor' and submits form

Normal Scenario

9. Visitor fills instructor application (name, email, department, credentials).
10. System stores application with status PENDING.
11. Registrar reviews and approves; no justification required for rejection.
12. Approved instructor receives unique ID and temporary password.

Exceptional Scenarios

- Rejection: Registrar clicks Reject; system marks application REJECTED; no justification stored.
- Duplicate email: Validation error returned.
- Department field empty: Form validation prevents submission.

UC-04: Manage Semester Phases (Petri Net Included)

Field	Details
Actor	Registrar
Precondition	System has at least one active semester record
Postcondition	Semester phase advanced with audit timestamp; phase-triggered rules executed
Trigger	Registrar selects 'Advance Phase' in semester control panel

Normal Scenario

13. Registrar opens Semester Management screen.
14. System displays current phase, timestamp of last transition, and next valid phase.
15. Registrar confirms the advance action.
16. System executes phase-transition logic (see Petri net below), writes audit event, updates semester phase.
17. Dashboard refreshes to show new phase and active controls.

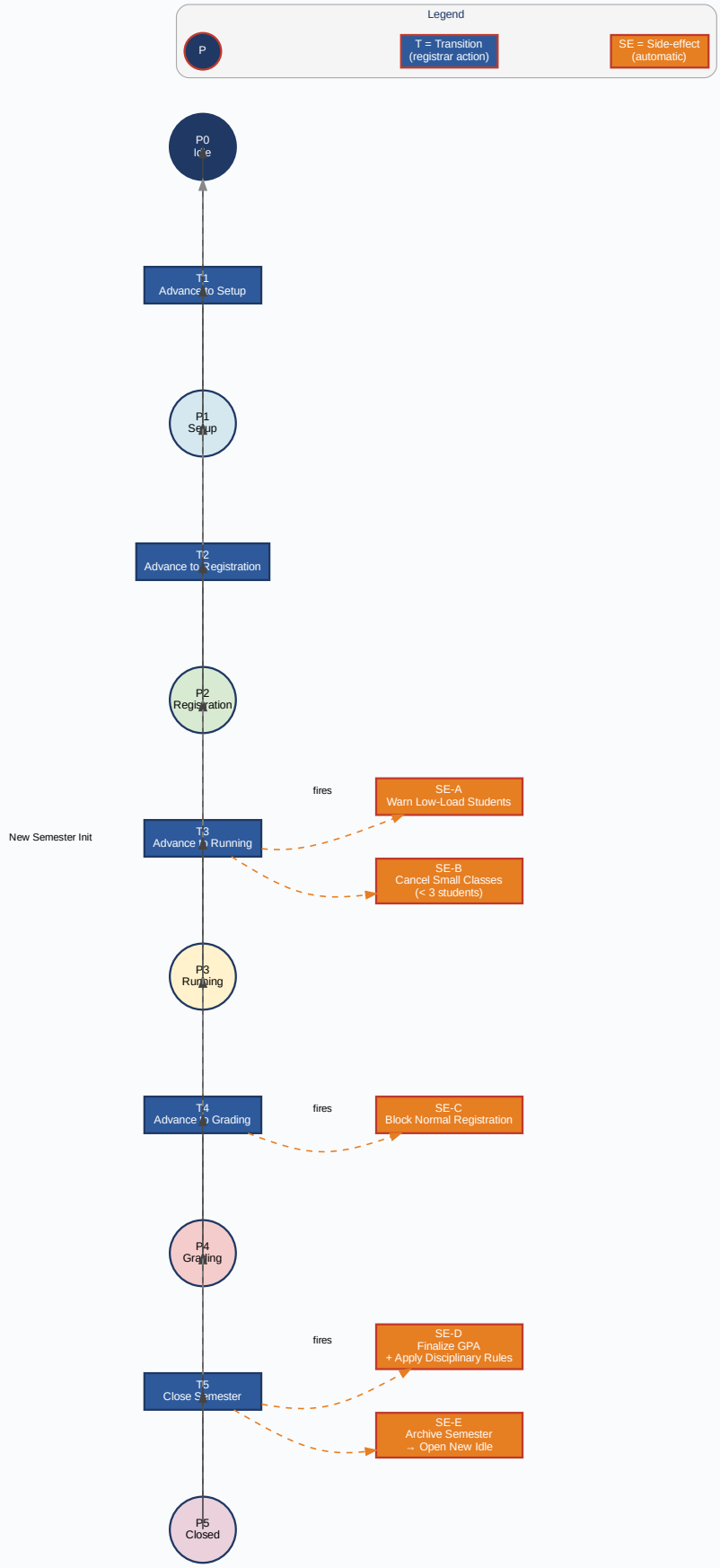
Exceptional Scenarios

- Attempt to skip phase (e.g., Setup → Running): System returns 400; no transition performed.
- Grading → next Setup: System finalizes GPA, triggers termination/warning rules, archives semester before opening new Setup.
- No courses defined in Setup: System warns registrar but allows advance (courses can be added after the fact, before Registration).

Petri Net — Semester Phase Lifecycle

```
PETRI NET — Semester Phase Transitions
═══════════════════════════════════════

Places (states):                      Transitions (registrar-triggered):
P0 = [Pre-Semester / Idle]            T1 = advance to Setup
P1 = [Setup Phase]                    T2 = advance to Registration
P2 = [Registration Phase]             T3 = advance to Running
P3 = [Running Phase]                  T4 = advance to Grading
P4 = [Grading Phase]                  T5 = close semester / archive
P5 = [Semester Closed]

Flow:
P0 ──[T1]──> P1 ──[T2]──> P2 ──[T3]──> P3 ──[T4]──> P4 ──[T5]──> P5
 ↑                                                                  │
 └──────────────────────── [New Semester Init] ──────────────────────┘

Side-effect arcs (inhibitor / automatic):
T3 fires → also fires: [WarnStudentsLowLoad], [CancelSmallClasses]
T4 fires → also fires: [BlockRegistration]
T5 fires → also fires: [FinalizeGPA], [TriggerDisciplinaryRules], [ArchiveSemester]

Token semantics: A token in Pk means the semester is currently in phase k.
Only one token exists at any time (mutually exclusive phases).
```

UC-04: Semester Phase Lifecycle — Petri Net



UC-05: Register Courses

Field	Details
Actor	Student
Precondition	Registration or special-registration phase active; student active and not suspended
Postcondition	Student enrolled or waitlisted in selected class sections
Trigger	Student opens Course Registration screen and submits selections

Normal Scenario

18. Student opens registration screen; system shows available classes with capacity and schedule.
19. Student selects 2–4 classes.
20. System validates: no schedule conflicts, count in range, class available for retake only if prior grade was F.
21. For each class: if capacity available → enrolled; else → placed on waitlist.
22. System returns summary of enrollments and waitlist positions.

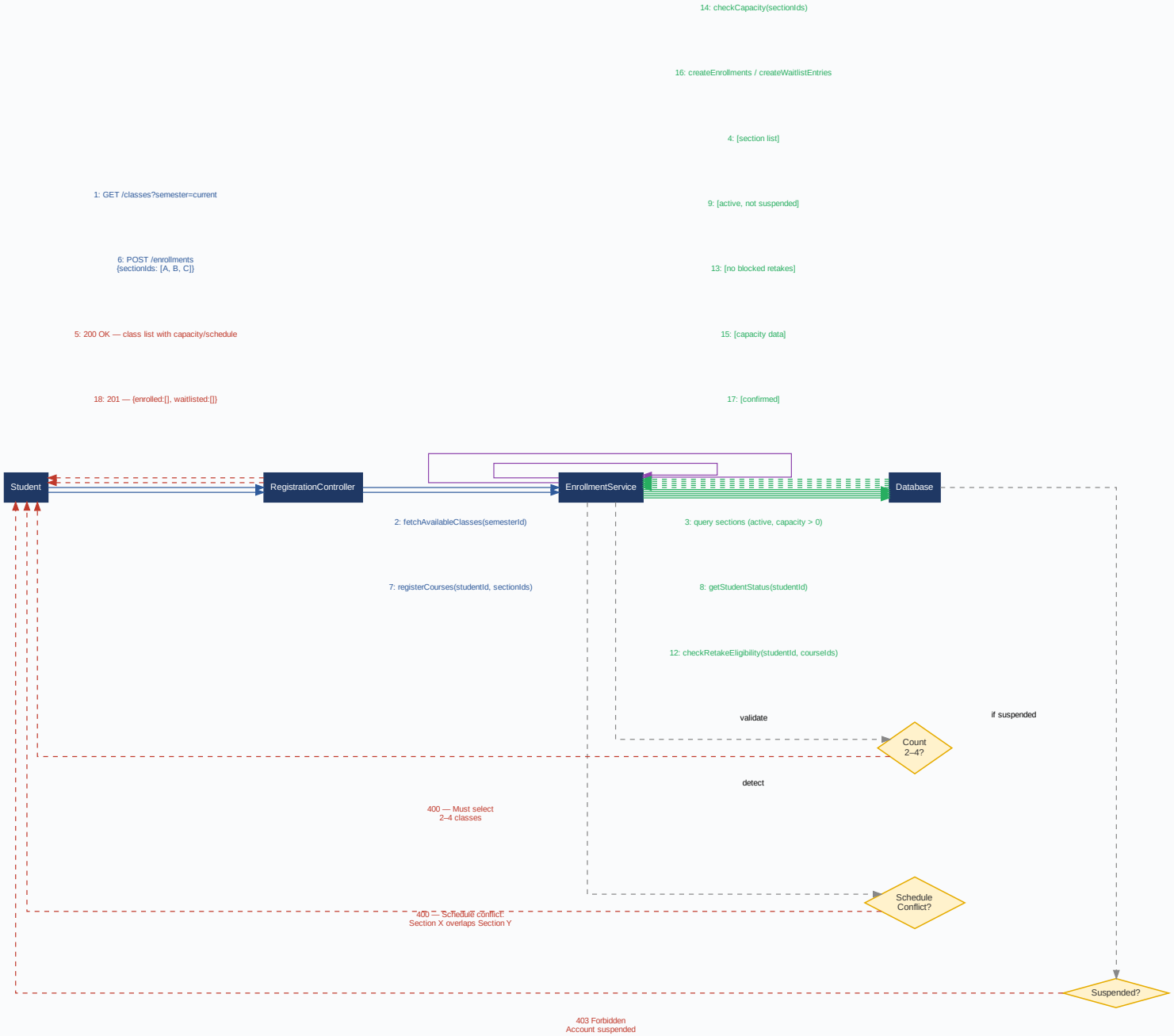
Exceptional Scenarios

- Fewer than 2 or more than 4 classes selected: Validation error; no enrollment performed.
- Time conflict detected: System highlights conflicting pair; student must resolve.
- Student suspended: Registration screen disabled with suspension notice.
- Student attempts retake of passed course: System blocks with message 'Retake only allowed for F grade'.
- Class full: Student automatically placed on waitlist; shown position number.

Sequence Diagram — UC-05

```
Student           RegistrationCtrl       EnrollmentSvc         DB
  |                       |                     |                 |
  |──1:GET /classes──────>|                     |                 |
  |                       |──2:fetchAvail──────>|                 |
  |                       |                     |──3:query───────>|
  |                       |                     |<──classes───────|
  |<──4:class list────────|                     |                 |
  |──5:POST /enroll──────>|                     |                 |
  |  {sectionIds:[a,b,c]} |                     |                 |
  |                       |──6:validateLoad─────>|                 |
  |                       |──7:checkConflicts──>|                 |
  |                       |──8:checkRetake──────>|                 |
  |                       |                     |──9:checkCap────>|
  |                       |                     |<──cap data──────|
  |                       |                     |──10:enroll/WL──>|
  |<──11:enrollment summary──────────────────────────────────────|
```

UC-05: Register Courses — Sequence Diagram



UC-06: Admit Waitlisted Student

Field	Details
Actor	Instructor
Precondition	Registration phase active (or registrar override); class has waitlisted students; instructor owns the class
Postcondition	Waitlisted student promoted to enrolled; capacity counter decremented
Trigger	Instructor opens waitlist screen and clicks 'Admit'

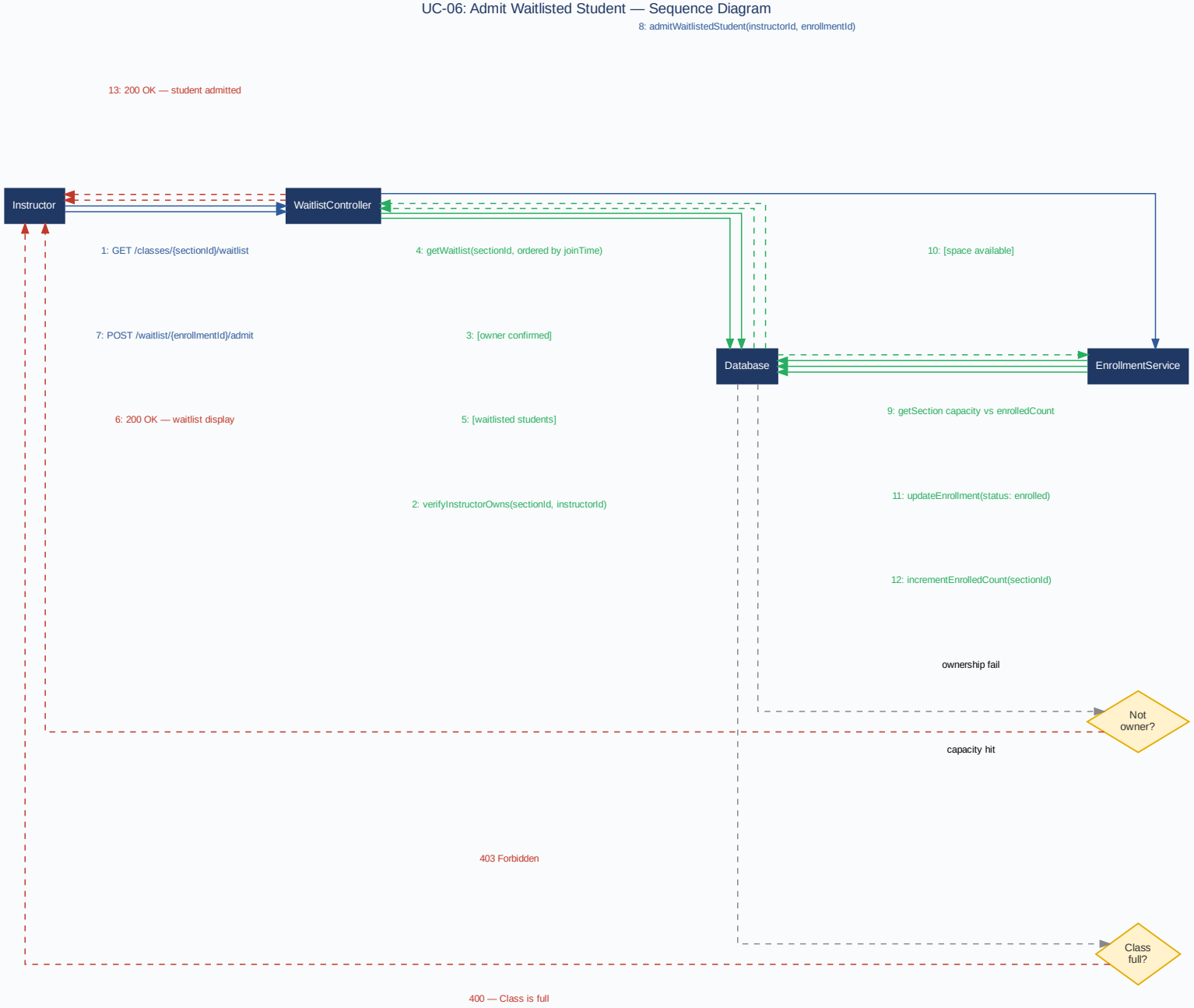
Normal Scenario

23. Instructor navigates to their class roster and selects the Waitlist tab.
24. System shows ordered waitlist with student names and join timestamps.
25. Instructor selects student and clicks Admit.
26. System verifies instructor owns the class, checks capacity, promotes student to Enrolled.

Exceptional Scenarios

- Class at capacity still: System rejects admission with 'Class is full'.
- Instructor not owner: API returns 403 Forbidden.
- Student on waitlist already dropped: System removes stale entry, shows next candidate.

UC-06: Admit Waitlisted Student — Sequence Diagram



UC-07: Submit Review (Petri Net Included)

Field	Details
Actor	Student
Precondition	Student enrolled in class; instructor has NOT yet posted final grades
Postcondition	Review stored; taboo moderation applied; warnings issued if applicable
Trigger	Student submits review form with star rating and text

Normal Scenario

27. Student navigates to enrolled class and clicks 'Leave Review'.
28. Student enters 1–5 star rating and optional text.
29. System checks: student enrolled, grades not yet posted.
30. System scans text for taboo words in the configured list.
31. 0 taboo words: publish normally.
32. 1–2 taboo words: publish with words masked (*), issue 1 warning to student.
33. ≥3 taboo words: do not publish, issue 2 warnings to student.

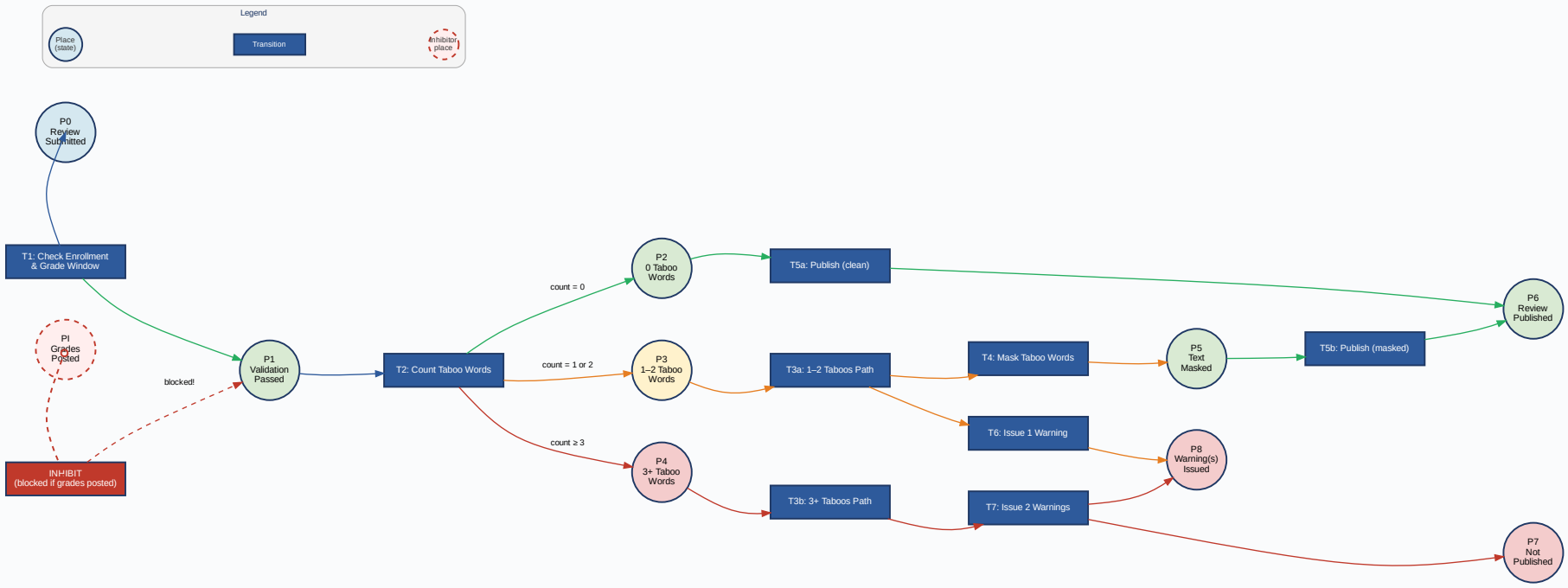
Exceptional Scenarios

- Attempt after grades posted: System returns 403 with message 'Review window closed'.
- Student not enrolled: 403 Forbidden.
- Rating out of range (not 1–5): Validation error.
- Third warning triggered: Suspension process initiated (see FR-021).

Petri Net — Review Moderation Flow

```
PETRI NET — Taboo Moderation Process
═══════════════════════════════════════

Places:                          Transitions:
P0 = [Review Submitted]          T1 = check enrollment & grade window
P1 = [Validation Passed]         T2 = count taboo words
P2 = [0 Taboo Words]             T3a = 1-2 taboos path
P3 = [1-2 Taboo Words]           T3b = 3+ taboos path
P4 = [3+ Taboo Words]            T4 = mask words
P5 = [Text Masked]               T5 = publish
P6 = [Published]                 T6 = issue 1 warning
P7 = [Not Published]             T7 = issue 2 warnings
P8 = [Warning(s) Issued]

Flow arcs:
P0 ──[T1]──> P1
P1 ──[T2]──> P2  (0 taboo)  ──[T5]──> P6
P1 ──[T2]──> P3  (1-2 taboo) ──[T4]──> P5 ──[T5]──> P6
                              └──[T6]──> P8
P1 ──[T2]──> P4  (3+ taboo)  ──[T7]──> P7
                              └──[T7]──> P8

Inhibitor arc: T1 is inhibited if [GradesPosted] place is marked.
```

UC-07: Review Submission & Taboo Moderation — Petri Net



UC-08: Submit Grades (Petri Net Included)

Field	Details
Actor	Instructor
Precondition	Grading phase active; instructor has enrolled students in class
Postcondition	All student grades recorded; class marked grading-complete; GPA triggers evaluated
Trigger	Instructor submits grade form for their class roster

Normal Scenario

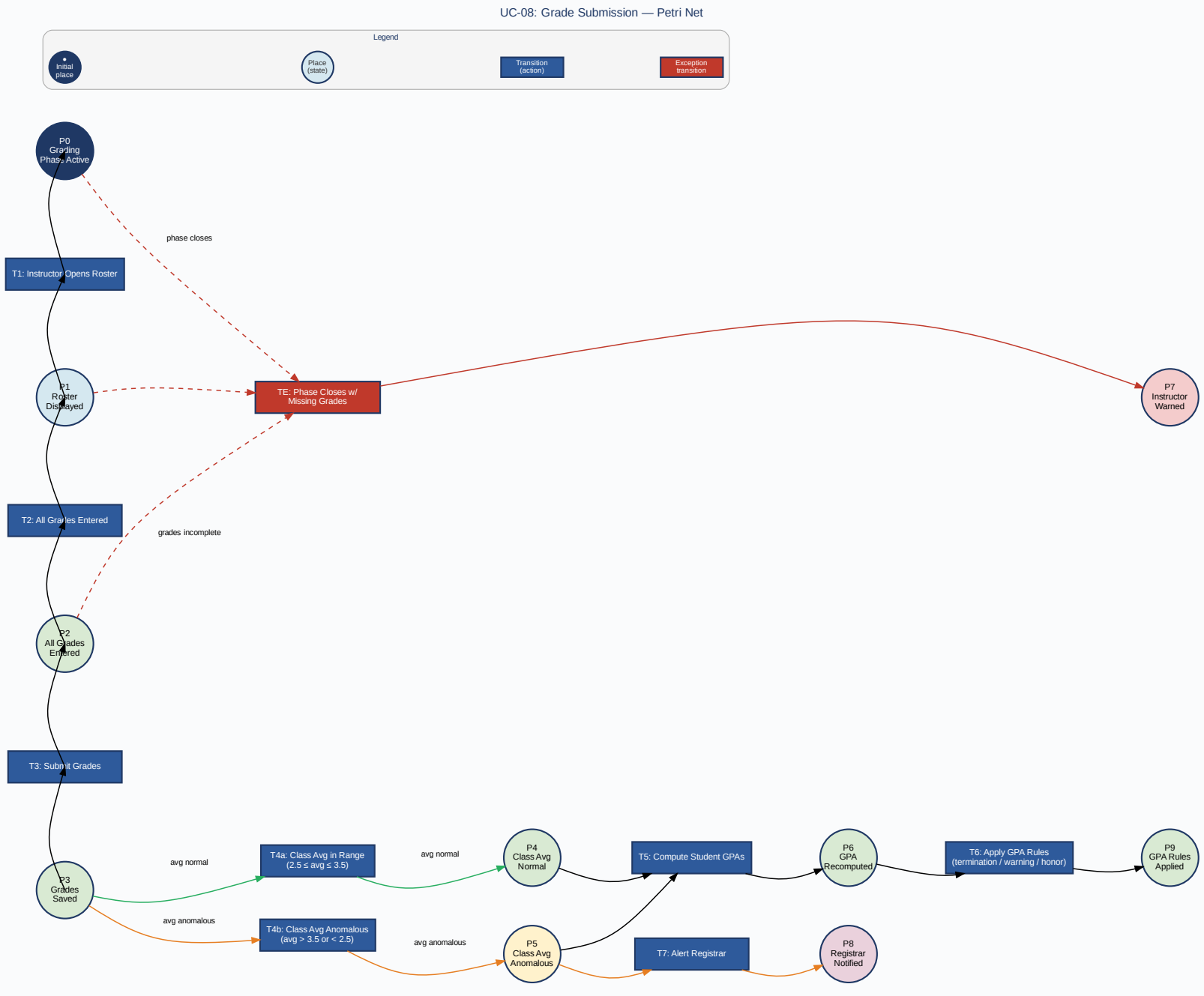
34. Instructor opens Grade Entry screen for their class.
35. System displays roster of enrolled students with grade input fields.
36. Instructor enters grade (A/B/C/D/F or numeric 0–100) for each student.
37. Instructor submits; system validates all entries present.
38. System saves grades, recomputes student GPAs, evaluates class average.
39. If class GPA > 3.5 or < 2.5, registrar receives automatic notification for review.

Exceptional Scenarios

- Phase closes with incomplete grades: System issues warning to instructor.
- Class average out of bounds: Registrar must question instructor; inadequate justification may lead to warning or firing.
- Instructor tries to edit after submission: System locks grade entry; registrar override required.

Petri Net — Grade Submission Flow

```
PETRI NET — Grade Submission
═══════════════════════════════════════


Places:                           Transitions:
P0 = [Grading Phase Active]       T1 = instructor opens roster
P1 = [Roster Displayed]           T2 = all grades entered
P2 = [Grades Entered]             T3 = submit
P3 = [Grades Saved]               T4 = evaluate class avg
P4 = [Class Avg Normal]           T5 = flag for registrar review
P5 = [Class Avg Anomalous]        T6 = compute student GPAs
P6 = [GPA Computed]               T7 = close phase w/missing grades
P7 = [Warn Instructor]            T8 = issue warning

Flow arcs:
P0 ──[T1]──> P1 ──[T2]──> P2 ──[T3]──> P3
P3 ──[T4]──> P4  (avg in range)  ──[T6]──> P6
P3 ──[T4]──> P5  (avg anomalous) ──[T5]──> notify registrar
                                  └──[T6]──> P6
P0+P1+P2 ──[T7: phase close w/missing]──> P7 ──[T8]──> warn instructor
```


UC-08: Grade Submission — Petri Net
Legend
Initial place
Place (state)
Transition (action)
Exception transition
P0 Grading Phase Active
phase closes
T1: Instructor Opens Roster
P1 Roster Displayed
TE: Phase Closes w/ Missing Grades
P7 Instructor Warned
T2: All Grades Entered
grades incomplete
P2 All Grades Entered
T3: Submit Grades
P3 Grades Saved
avg normal
T4a: Class Avg in Range (2.5 ≤ avg ≤ 3.5)
avg normal
P4 Class Avg Normal
T5: Compute Student GPAs
P6 GPA Recomputed
T6: Apply GPA Rules (termination / warning / honor)
P9 GPA Rules Applied
avg anomalous
T4b: Class Avg Anomalous (avg > 3.5 or < 2.5)
avg anomalous
P5 Class Avg Anomalous
T7: Alert Registrar
P8 Registrar Notified

UC-09: File Complaint

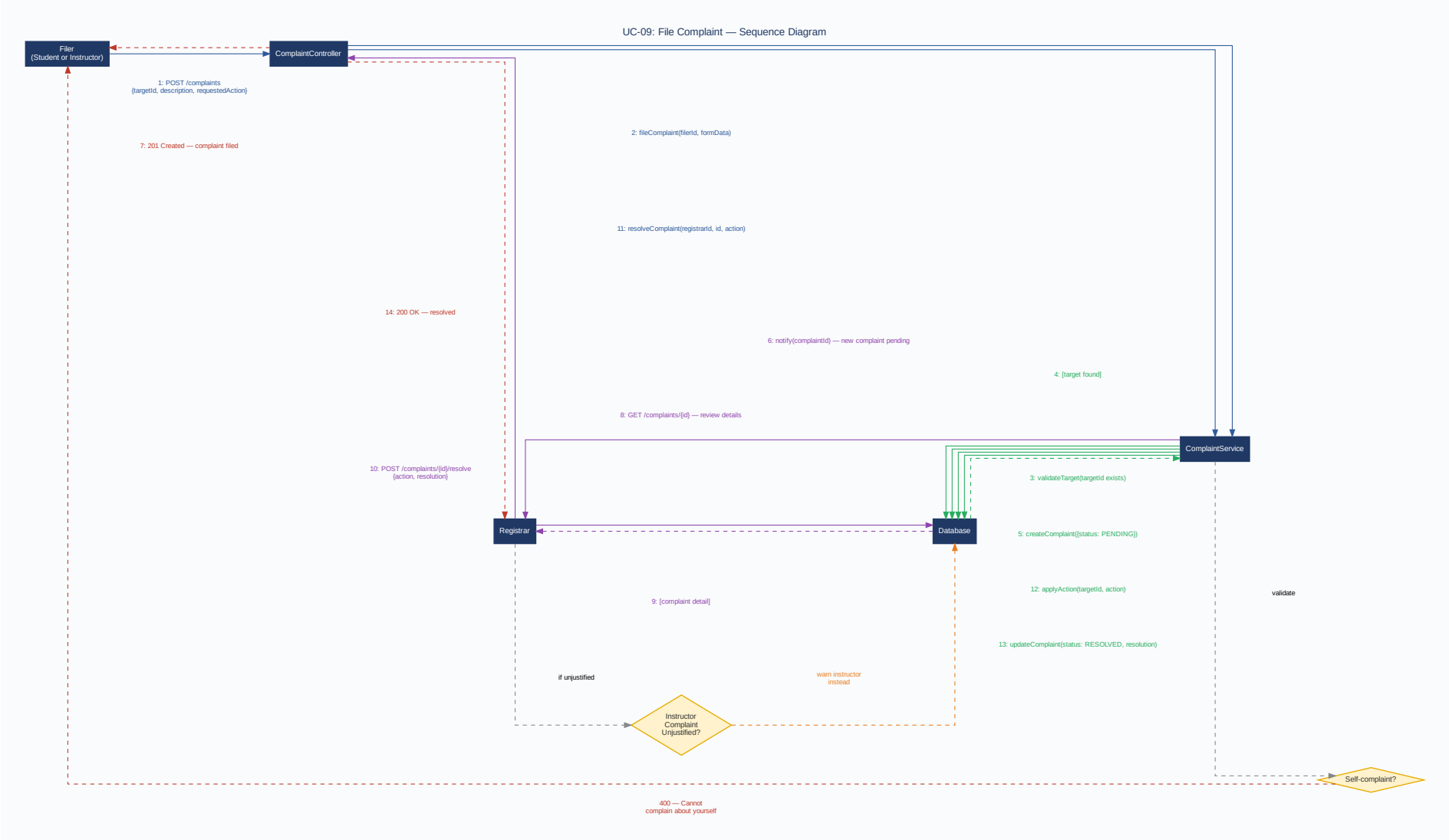
Field	Details
Actor	Student or Instructor (filer), Registrar (resolver)
Precondition	User is authenticated; target user exists
Postcondition	Complaint resolved; punishment or counter-action applied
Trigger	User clicks 'File Complaint' and submits complaint form

Normal Scenario

40. Filer selects target (student or instructor), enters description, specifies desired action.
41. System stores complaint as PENDING and notifies registrar.
42. Registrar reviews complaint details, reviews evidence.
43. Registrar applies action: warn/suspend/de-register target, or if instructor complaint unjustified, warn the instructor.
44. System records decision and applies consequences.

Exceptional Scenarios

- Registrar takes no action: System does not auto-close; complaint remains PENDING until resolved.
- Filer files against themselves: Validation prevents self-complaint.
- Target already suspended: Registrar warned; additional penalty may still be recorded.


UC-09: File Complaint — Sequence Diagram
Filer
(Student or Instructor)
ComplaintController
ComplaintService
Registrar
Database
1: POST /complaints
{targetId, description, requestedAction}
2: fileComplaint(filerId, formData)
3: validateTarget(targetId exists)
4: [target found]
5: createComplaint({status: PENDING})
6: notify(complaintId) — new complaint pending
7: 201 Created — complaint filed
8: GET /complaints/{id} — review details
9: [complaint detail]
10: POST /complaints/{id}/resolve
{action, resolution}
11: resolveComplaint(registrarId, id, action)
12: applyAction(targetId, action)
13: updateComplaint(status: RESOLVED, resolution)
14: 200 OK — resolved
validate
Self-complaint?
400 — Cannot
complain about yourself
if unjustified
Instructor
Complaint
Unjustified?
warn instructor
instead

UC-10: Ask AI Assistant

Field	Details
Actor	Any authenticated user (or Visitor for general questions)
Precondition	User is on AI Assistant page; AI subsystem is available
Postcondition	User receives grounded answer or LLM fallback with warning
Trigger	User types a question and submits

Normal Scenario

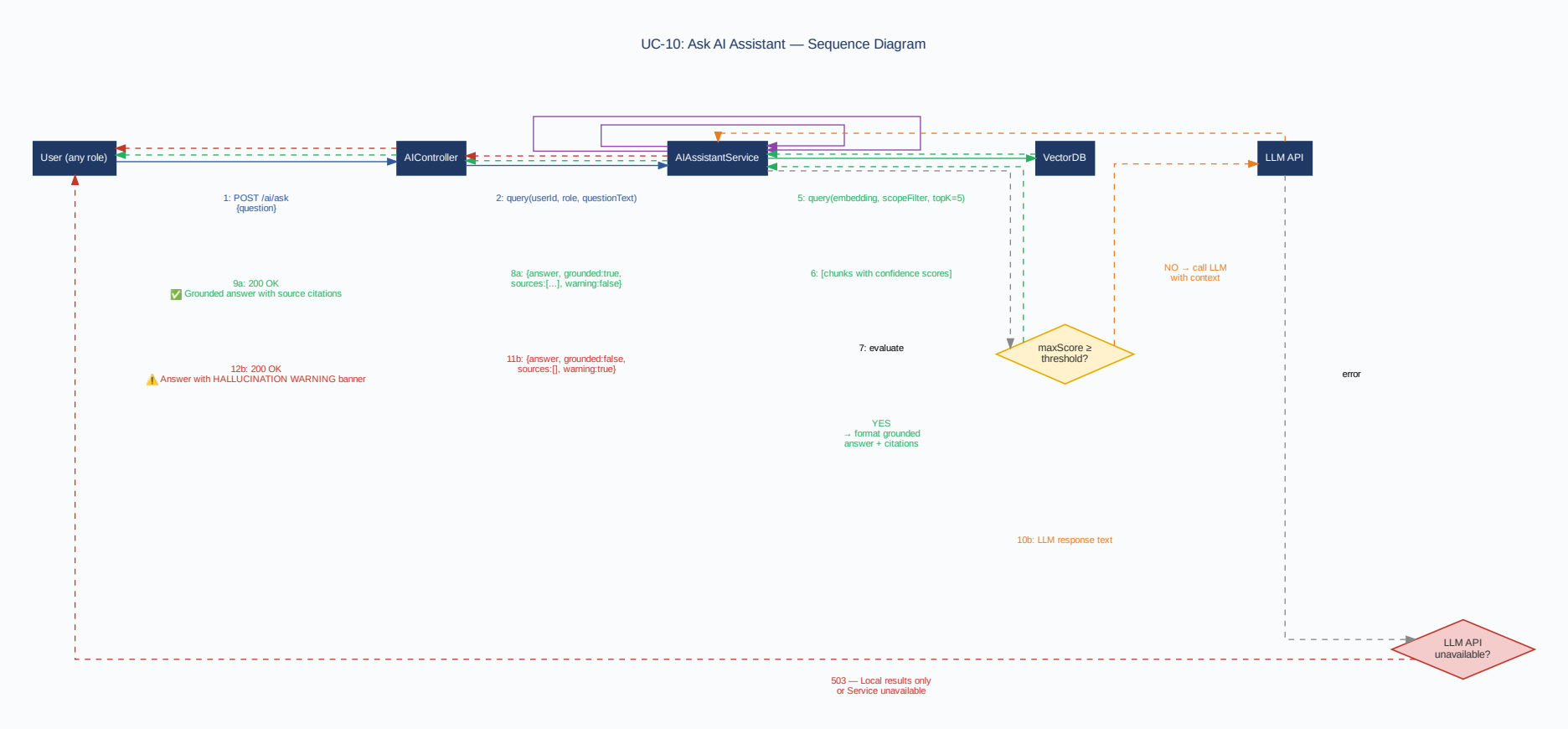
45. User types question in AI chat input.
46. System builds role-scoped retrieval filter (visitor: general only; student: + enrolled class info; instructor: + own student info).
47. System queries vector DB with filtered scope; retrieves top-k matching chunks.
48. If retrieval confidence ≥ threshold: return grounded answer with source citations.
49. If confidence < threshold: call LLM API with context; display answer with hallucination warning banner.

Exceptional Scenarios

- LLM API unavailable: System returns local retrieval result only, or 'Service temporarily unavailable'.
- No relevant chunks found: Fallback immediately to LLM with warning.
- User asks out-of-scope question (e.g., student asking about another student's grades): Role filter prevents retrieval; answer not provided.

Sequence Diagram — UC-10

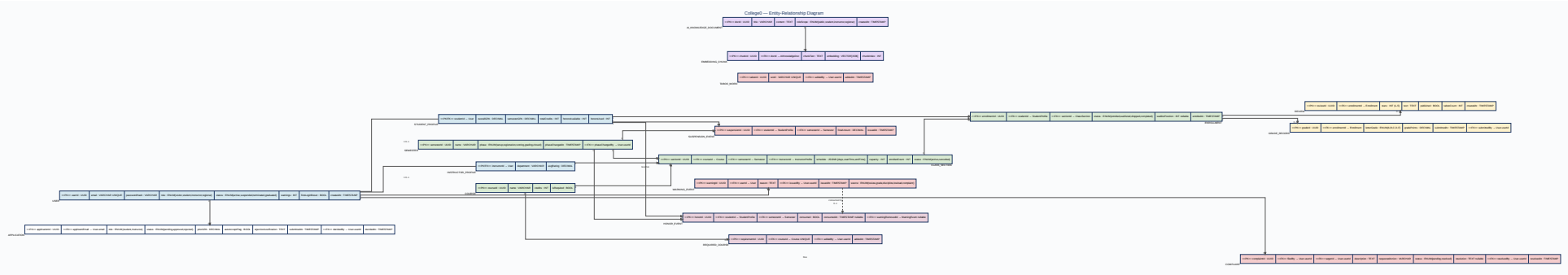
```
User            AIController       VectorDB        LLM API
  |                 |                 |                |
  |--1:POST /ai/ask-->|               |                |
  |                 |--2:buildFilter-->               |
  |                 |--3:query-------->|              |
  |                 |<--4:chunks+score-|              |
  |                 |                 |                |
  |          [score >= threshold?]     |               |
  |                 |--YES: format answer             |
  <--5a:grounded ans-|  with citations                |
  |                 |--NO------------------------------>|
  |                 |                                 |
  |                 |<--6:LLM response----------------|
  <--5b:ans+WARN----|                                 |
```


UC-10: Ask AI Assistant — Sequence Diagram
User (any role)
AIController
AIAssistantService
VectorDB
LLM API
1: POST /ai/ask
{question}
2: query(userId, role, questionText)
5: query(embedding, scopeFilter, topK=5)
6: [chunks with confidence scores]
7: evaluate
maxScore ≥
threshold?
NO → call LLM
with context
YES
→ format grounded
answer + citations
10b: LLM response text
error
LLM API
unavailable?
8a: {answer, grounded:true,
sources:[...], warning:false}
11b: {answer, grounded:false,
sources:[], warning:true}
9a: 200 OK
✅ Grounded answer with source citations
12b: 200 OK
⚠️ Answer with HALLUCINATION WARNING banner
503 — Local results only
or Service unavailable

3. Entity-Relationship Diagram

The E-R diagram below captures all persistent entities, their attributes (primary and foreign keys marked), and relationship cardinalities for the College0 system.

```
 ENTITY-RELATIONSHIP DIAGRAM — College0
 ═══════════════════════════════════════════════════════════

[USER]──────────────────────────────────────────────────
 PK: userId (UUID)
 email (UNIQUE), passwordHash, role (ENUM: visitor/student/instructor/registrar)
 status (ENUM: active/suspended/terminated/graduated)
 warnings (INT, default 0), firstLoginReset (BOOL)
 createdAt, updatedAt

[APPLICATION]───────────────────────────────────────────
 PK: applicationId (UUID)
 FK: applicantEmail → USER.email
 role (student/instructor), status (pending/approved/rejected)
 priorGPA (DECIMAL), personalStatement (TEXT)
 autoAcceptFlag (BOOL), rejectionJustification (TEXT, nullable)
 submittedAt, decidedAt, decidedBy (FK→USER.userId)

[STUDENT_PROFILE]───────────────────────────────────────
 PK/FK: studentId → USER.userId (1:1)
 overallGPA (DECIMAL), semesterGPA (DECIMAL)
 totalCredits (INT), honorsAvailable (INT), honorsUsed (INT)

[INSTRUCTOR_PROFILE]────────────────────────────────────
 PK/FK: instructorId → USER.userId (1:1)
 department (VARCHAR), avgRating (DECIMAL)

[SEMESTER]──────────────────────────────────────────────
 PK: semesterId (UUID)
 name (VARCHAR), phase (ENUM: setup/registration/running/grading/closed)
 phaseChangedAt (TIMESTAMP), phaseChangedBy (FK→USER.userId)

[COURSE]────────────────────────────────────────────────
 PK: courseId (UUID)
 name (VARCHAR), credits (INT), isRequired (BOOL)

[CLASS_SECTION]─────────────────────────────────────────
 PK: sectionId (UUID)
 FK: courseId → COURSE.courseId
 FK: semesterId → SEMESTER.semesterId
 FK: instructorId → INSTRUCTOR_PROFILE.instructorId
 schedule (JSONB: {days[], startTime, endTime})
 capacity (INT), enrolledCount (INT)
 status (ENUM: active/cancelled)

[ENROLLMENT]────────────────────────────────────────────
 PK: enrollmentId (UUID)
 FK: studentId → STUDENT_PROFILE.studentId
 FK: sectionId → CLASS_SECTION.sectionId
 status (ENUM: enrolled/waitlisted/dropped/completed)
 waitlistPosition (INT, nullable)
 enrolledAt (TIMESTAMP)

[GRADE_RECORD]──────────────────────────────────────────
 PK: gradeId (UUID)
 FK: enrollmentId → ENROLLMENT.enrollmentId (1:1)
 letterGrade (ENUM: A/B/C/D/F), gradePoints (DECIMAL)
 submittedAt, submittedBy (FK→USER.userId)

[REVIEW]────────────────────────────────────────────────
 PK: reviewId (UUID)
 FK: enrollmentId → ENROLLMENT.enrollmentId
 stars (INT: 1-5), text (TEXT)
```


CollegeO — Entity-Relationship Diagram
id INTEGER NOT NULL
name VARCHAR(255) NOT NULL
email VARCHAR(255) NOT NULL
password_hash VARCHAR(255) NOT NULL
role ENUM('student','instructor','admin') NOT NULL
status ENUM('active','suspended','deactivated') DEFAULT 'active'
avatar_path VARCHAR(255) NULL
bio TEXT NULL
email_verified_at DATETIME NULL
last_login_at DATETIME NULL
last_login_ip VARCHAR(45) NULL
created_at DATETIME NOT NULL
updated_at DATETIME NOT NULL
deleted_at DATETIME NULL
id INTEGER NOT NULL
user_id INTEGER NOT NULL
expertise VARCHAR(255) NULL
qualifications TEXT NULL
years_experience INTEGER NULL
hourly_rate DECIMAL(8,2) NULL
rating_avg DECIMAL(3,2) NOT NULL DEFAULT 0.00
rating_count INTEGER NOT NULL DEFAULT 0
students_count INTEGER NOT NULL DEFAULT 0
courses_count INTEGER NOT NULL DEFAULT 0
is_verified BOOLEAN NOT NULL DEFAULT FALSE
verified_at DATETIME NULL
created_at DATETIME NOT NULL
updated_at DATETIME NOT NULL
id INTEGER NOT NULL
slug VARCHAR(255) NOT NULL
name VARCHAR(255) NOT NULL
description TEXT NULL
icon VARCHAR(255) NULL
color VARCHAR(7) NULL
parent_id INTEGER NULL
sort_order INTEGER NOT NULL DEFAULT 0
is_active BOOLEAN NOT NULL DEFAULT TRUE
created_at DATETIME NOT NULL
updated_at DATETIME NOT NULL
id INTEGER NOT NULL
instructor_id INTEGER NOT NULL
category_id INTEGER NULL
title VARCHAR(255) NOT NULL
slug VARCHAR(255) NOT NULL
short_description VARCHAR(500) NULL
description TEXT NULL
what_you_learn JSON NULL
requirements JSON NULL
target_audience JSON NULL
thumbnail_path VARCHAR(255) NULL
preview_video_path VARCHAR(255) NULL
level ENUM('beginner','intermediate','advanced','all') NOT NULL DEFAULT 'all'
language VARCHAR(10) NOT NULL DEFAULT 'en'
price DECIMAL(8,2) NOT NULL DEFAULT 0.00
discount_price DECIMAL(8,2) NULL
discount_ends_at DATETIME NULL
is_free BOOLEAN NOT NULL DEFAULT FALSE
is_featured BOOLEAN NOT NULL DEFAULT FALSE
is_bestseller BOOLEAN NOT NULL DEFAULT FALSE
is_certificate BOOLEAN NOT NULL DEFAULT FALSE
rating_avg DECIMAL(3,2) NOT NULL DEFAULT 0.00
rating_count INTEGER NOT NULL DEFAULT 0
students_count INTEGER NOT NULL DEFAULT 0
lessons_count INTEGER NOT NULL DEFAULT 0
duration_minutes INTEGER NOT NULL DEFAULT 0
status ENUM('draft','pending','approved','rejected') NOT NULL DEFAULT 'draft'
rejection_reason TEXT NULL
published_at DATETIME NULL
created_at DATETIME NOT NULL
updated_at DATETIME NOT NULL
deleted_at DATETIME NULL
id INTEGER NOT NULL
course_id INTEGER NOT NULL
instructor_id INTEGER NOT NULL
title VARCHAR(255) NOT NULL
description TEXT NULL
sort_order INTEGER NOT NULL DEFAULT 0
is_published BOOLEAN NOT NULL DEFAULT FALSE
created_at DATETIME NOT NULL
updated_at DATETIME NOT NULL
deleted_at DATETIME NULL
id INTEGER NOT NULL
section_id INTEGER NOT NULL
course_id INTEGER NOT NULL
instructor_id INTEGER NOT NULL
title VARCHAR(255) NOT NULL
slug VARCHAR(255) NOT NULL
type ENUM('video','text','quiz','assignment','live') NOT NULL
content JSON NULL
video_path VARCHAR(255) NULL
video_duration INTEGER NULL
video_size_mb DECIMAL(8,2) NULL
video_thumbnail_path VARCHAR(255) NULL
is_preview BOOLEAN NOT NULL DEFAULT FALSE
is_required BOOLEAN NOT NULL DEFAULT TRUE
sort_order INTEGER NOT NULL DEFAULT 0
created_at DATETIME NOT NULL
updated_at DATETIME NOT NULL
deleted_at DATETIME NULL
id INTEGER NOT NULL
user_id INTEGER NOT NULL
course_id INTEGER NOT NULL
enrolled_at DATETIME NOT NULL
expiry_date DATETIME NULL
progress_percent DECIMAL(5,2) NOT NULL DEFAULT 0.00
completed_lessons INTEGER NOT NULL DEFAULT 0
total_lessons INTEGER NOT NULL DEFAULT 0
last_accessed_at DATETIME NULL
completed_at DATETIME NULL
certificate_issued BOOLEAN NOT NULL DEFAULT FALSE
certificate_issued_at DATETIME NULL
rating INTEGER NULL
review TEXT NULL
reviewed_at DATETIME NULL
created_at DATETIME NOT NULL
updated_at DATETIME NOT NULL
deleted_at DATETIME NULL
id INTEGER NOT NULL
user_id INTEGER NOT NULL
course_id INTEGER NOT NULL
lesson_id INTEGER NULL
progress_percent DECIMAL(5,2) NOT NULL DEFAULT 0.00
watched_seconds INTEGER NOT NULL DEFAULT 0
completed_at DATETIME NULL
created_at DATETIME NOT NULL
updated_at DATETIME NOT NULL
id INTEGER NOT NULL
user_id INTEGER NOT NULL
course_id INTEGER NOT NULL
coupon_id INTEGER NULL
order_id INTEGER NOT NULL
price_paid DECIMAL(8,2) NOT NULL DEFAULT 0.00
payment_method VARCHAR(50) NULL
payment_id VARCHAR(255) NULL
status ENUM('active','expired','refunded') NOT NULL DEFAULT 'active'
created_at DATETIME NOT NULL
updated_at DATETIME NOT NULL
deleted_at DATETIME NULL
id INTEGER NOT NULL
code VARCHAR(50) NOT NULL
description VARCHAR(255) NULL
type ENUM('percentage','fixed') NOT NULL
value DECIMAL(8,2) NOT NULL
min_order_amount DECIMAL(8,2) NULL
max_discount DECIMAL(8,2) NULL
usage_limit INTEGER NULL
usage_per_user INTEGER NOT NULL DEFAULT 1
used_count INTEGER NOT NULL DEFAULT 0
applicable_to ENUM('all','courses','categories') NOT NULL DEFAULT 'all'
applicable_ids JSON NULL
starts_at DATETIME NULL
expires_at DATETIME NULL
is_active BOOLEAN NOT NULL DEFAULT TRUE
created_at DATETIME NOT NULL
updated_at DATETIME NOT NULL
deleted_at DATETIME NULL
id INTEGER NOT NULL
coupon_id INTEGER NOT NULL
user_id INTEGER NOT NULL
order_id INTEGER NULL
discount_amount DECIMAL(8,2) NOT NULL
used_at DATETIME NOT NULL
id INTEGER NOT NULL
order_id INTEGER NOT NULL
user_id INTEGER NOT NULL
course_id INTEGER NOT NULL
price DECIMAL(8,2) NOT NULL DEFAULT 0.00
discount_amount DECIMAL(8,2) NOT NULL DEFAULT 0.00
tax_amount DECIMAL(8,2) NOT NULL DEFAULT 0.00
total_amount DECIMAL(8,2) NOT NULL DEFAULT 0.00
created_at DATETIME NOT NULL
updated_at DATETIME NOT NULL
deleted_at DATETIME NULL
id INTEGER NOT NULL
user_id INTEGER NOT NULL
order_number VARCHAR(20) NOT NULL
subtotal DECIMAL(8,2) NOT NULL DEFAULT 0.00
discount_amount DECIMAL(8,2) NOT NULL DEFAULT 0.00
tax_amount DECIMAL(8,2) NOT NULL DEFAULT 0.00
total_amount DECIMAL(8,2) NOT NULL DEFAULT 0.00
currency VARCHAR(3) NOT NULL DEFAULT 'USD'
status ENUM('pending','processing','completed','cancelled','refunded') NOT NULL DEFAULT 'pending'
payment_method VARCHAR(50) NULL
payment_id VARCHAR(255) NULL
paid_at DATETIME NULL
cancelled_at DATETIME NULL
cancel_reason TEXT NULL
created_at DATETIME NOT NULL
updated_at DATETIME NOT NULL
deleted_at DATETIME NULL
id INTEGER NOT NULL
name VARCHAR(255) NOT NULL
email VARCHAR(255) NOT NULL
password_hash VARCHAR(255) NOT NULL
role ENUM('admin','moderator') NOT NULL
status ENUM('active','suspended') NOT NULL DEFAULT 'active'
avatar_path VARCHAR(255) NULL
last_login_at DATETIME NULL
last_login_ip VARCHAR(45) NULL
email_verified_at DATETIME NULL
created_at DATETIME NOT NULL
updated_at DATETIME NOT NULL
deleted_at DATETIME NULL
id INTEGER NOT NULL
application_id INTEGER NOT NULL
reviewer_id INTEGER NOT NULL
action ENUM('approved','rejected') NOT NULL
comment TEXT NULL
created_at DATETIME NOT NULL
updated_at DATETIME NOT NULL
deleted_at DATETIME NULL
id INTEGER NOT NULL
user_id INTEGER NOT NULL
type ENUM('enrollment','payment','review','certificate','system') NOT NULL
title VARCHAR(255) NOT NULL
message TEXT NOT NULL
data JSON NULL
is_read BOOLEAN NOT NULL DEFAULT FALSE
read_at DATETIME NULL
created_at DATETIME NOT NULL
updated_at DATETIME NOT NULL
deleted_at DATETIME NULL
id INTEGER NOT NULL
user_id INTEGER NOT NULL
action VARCHAR(255) NOT NULL
model_type VARCHAR(255) NULL
model_id INTEGER NULL
old_values JSON NULL
new_values JSON NULL
ip_address VARCHAR(45) NULL
user_agent VARCHAR(255) NULL
created_at DATETIME NOT NULL
updated_at DATETIME NOT NULL
deleted_at DATETIME NULL
id INTEGER NOT NULL
user_id INTEGER NOT NULL
token VARCHAR(255) NOT NULL
expiry_date DATETIME NOT NULL
revoked BOOLEAN NOT NULL DEFAULT FALSE
created_at DATETIME NOT NULL
updated_at DATETIME NOT NULL
deleted_at DATETIME NULL
id INTEGER NOT NULL
user_id INTEGER NOT NULL
token VARCHAR(255) NOT NULL
expiry_date DATETIME NOT NULL

Entity A	Cardinality	Entity B	Notes
ENROLLMENT	1 : 0..1	REVIEW	Each enrollment allows one review
CLASS_SECTION	N : 1	COURSE	Many sections per course per semester
CLASS_SECTION	N : 1	SEMESTER	Sections belong to one semester
CLASS_SECTION	N : 1	INSTRUCTOR_PROFILE	Instructor teaches many sections
USER	1 : N	WARNING_EVENT	User accumulates many warnings
AI_KNOWLEDGE_DOCUMENT	1 : N	EMBEDDING_CHUNK	Document split into vector chunks

4. Detailed Design — Method Pseudo-code

This section provides pseudo-code for every significant system method, organized by service module. Each entry specifies: inputs, outputs, preconditions, and step-by-step logic.

4.1 AuthService

login(email, password) → JWT token

```
INPUT:  email (string), password (string)
OUTPUT: { token: JWT, firstLogin: bool } OR AuthError

1. user ← DB.findUser(email)
2. IF user is null THEN raise NotFoundError('Invalid credentials')
3. IF NOT bcrypt.verify(password, user.passwordHash) THEN raise AuthError
4. IF user.status IN [suspended, terminated] THEN raise ForbiddenError
5. payload ← { userId, role, exp: now+JWT_EXPIRES_IN }
6. token ← JWT.sign(payload, JWT_SECRET)
7. RETURN { token, firstLogin: user.firstLoginReset }
```

resetPassword(userId, newPassword) → void

```
INPUT:  userId (UUID), newPassword (string)
OUTPUT: void OR ValidationError

1. IF newPassword.length < 8 THEN raise ValidationError
2. hash ← bcrypt.hash(newPassword, saltRounds=12)
3. DB.updateUser(userId, { passwordHash: hash, firstLoginReset: false })
4. SessionStore.invalidateAll(userId)  // force re-login
```

4.2 ApplicationService

submitStudentApplication(formData) → Application

```
INPUT:  { name, email, priorGPA, personalStatement }
OUTPUT: Application record OR ValidationError

1. VALIDATE email format; IF duplicate THEN raise ConflictError
2. quota ← DB.countActiveStudents()
3. autoAccept ← (formData.priorGPA > 3.0) AND (quota < MAX_STUDENTS)
4. app ← DB.createApplication({
     ...formData, role:'student',
     status:'pending', autoAcceptFlag: autoAccept,
     submittedAt: now() })
5. RETURN app
```

decideApplication(registrarId, appId, decision, justification) → void

```
INPUT:  registrarId (UUID), appId (UUID),
        decision (ENUM: approved|rejected), justification (string, optional)
OUTPUT: void OR PolicyError

1. app ← DB.getApplication(appId)
2. IF decision == 'rejected' AND app.autoAcceptFlag == true
   THEN IF justification is empty THEN raise PolicyError('Justification required')
3. IF decision == 'approved'
```

```
   THEN user ← DB.createUser({ email:app.email, role:app.role,
          status:'active', firstLoginReset:true })
        tempPwd ← generateTempPassword()
        user.passwordHash ← bcrypt.hash(tempPwd)
        DB.saveUser(user)
        IF app.role == 'student' THEN DB.createStudentProfile(user.userId)
        ELSE DB.createInstructorProfile(user.userId)
        Notifier.sendCredentials(user.email, tempPwd)
4. DB.updateApplication(appId, {
     status: decision, decidedAt: now(),
     decidedBy: registrarId, rejectionJustification: justification })
```

4.3 SemesterService

advancePhase(registrarId, semesterId) → Semester

```
INPUT:  registrarId (UUID), semesterId (UUID)
OUTPUT: updated Semester OR PolicyError

1. sem ← DB.getSemester(semesterId)
2. nextPhase ← phaseOrder[sem.phase + 1]  // strict sequence
3. IF nextPhase is undefined THEN raise PolicyError('No next phase')
4. IF nextPhase == 'running'
   THEN runningPhaseSetup(semesterId)  // warn low-load students, cancel small classes
5. IF nextPhase == 'closed'
   THEN finalizeGrades(semesterId)     // GPA, terminations, honors
6. DB.updateSemester(semesterId, {
     phase: nextPhase, phaseChangedAt: now(), phaseChangedBy: registrarId })
7. AuditLog.record(registrarId, 'PHASE_ADVANCE', {from:sem.phase, to:nextPhase})
8. RETURN updated semester
```

runningPhaseSetup(semesterId) → void

```
INPUT:  semesterId (UUID)
OUTPUT: void (side effects: warnings, cancellations, special-reg windows)

1. students ← DB.getActiveStudents(semesterId)
2. FOR each student IN students
   activeCount ← DB.countActiveEnrollments(student.id, semesterId)
   IF activeCount < 2 THEN WarningService.issue(student.id, 'Low course load')
3. sections ← DB.getSections(semesterId, status='active')
4. FOR each section IN sections
   IF section.enrolledCount < 3
   THEN DB.updateSection(section.id, { status: 'cancelled' })
        affectedStudents ← DB.getEnrolledStudents(section.id)
        FOR each s IN affectedStudents
            DB.updateEnrollment(s.enrollmentId, { status: 'dropped' })
            SpecialRegService.openWindow(s.studentId, semesterId)
        WarningService.issue(section.instructorId, 'Class cancelled')
        IF DB.countActiveClasses(section.instructorId, semesterId) == 0
            SuspensionService.scheduleNextSemester(section.instructorId)
```

4.4 EnrollmentService

registerCourses(studentId, sectionIds[]) → EnrollmentResult

```
INPUT:  studentId (UUID), sectionIds (UUID[])
OUTPUT: { enrolled: Section[], waitlisted: Section[] } OR PolicyError
```

```
1. student ← DB.getStudentProfile(studentId)
2. IF student.status == 'suspended' THEN raise ForbiddenError
3. sem ← DB.getCurrentSemester()
4. IF sem.phase NOT IN [registration, special_registration]
   THEN raise PolicyError('Registration closed')
5. IF specialReg AND student NOT IN specialRegEligible(semesterId)
   THEN raise PolicyError('Special registration not available to you')
6. IF sectionIds.length < 2 OR sectionIds.length > 4
   THEN raise PolicyError('Must select 2–4 courses')
7. sections ← DB.getSections(sectionIds)
8. detectConflicts(sections)  // raise ConflictError if any overlap
9. FOR each section IN sections
   checkRetake(studentId, section.courseId)  // raise if passed already
10. enrolled ← [], waitlisted ← []
11. FOR each section IN sections
    IF section.enrolledCount < section.capacity
        DB.createEnrollment({studentId, sectionId, status:'enrolled'})
        DB.incrementEnrolled(section.id)
        enrolled.append(section)
    ELSE
        pos ← DB.countWaitlisted(section.id) + 1
        DB.createEnrollment({studentId, sectionId, status:'waitlisted',
waitlistPosition:pos})
        waitlisted.append(section)
12. RETURN { enrolled, waitlisted }
```

detectConflicts(sections[]) → void

```
INPUT:  sections (Section[]) each with schedule: {days[], startTime, endTime}
OUTPUT: void OR ConflictError(pair)

1. FOR i FROM 0 TO sections.length - 1
   FOR j FROM i+1 TO sections.length - 1
      a ← sections[i].schedule
      b ← sections[j].schedule
      sharedDays ← intersection(a.days, b.days)
      IF sharedDays is NOT empty
         IF a.startTime < b.endTime AND b.startTime < a.endTime
            THEN raise ConflictError(sections[i], sections[j])
```

4.5 ReviewService

submitReview(studentId, enrollmentId, stars, text) → Review

```
INPUT:  studentId (UUID), enrollmentId (UUID), stars (1-5), text (string)
OUTPUT: Review OR PolicyError

1. enrollment ← DB.getEnrollment(enrollmentId)
2. IF enrollment.studentId != studentId THEN raise ForbiddenError
3. IF DB.gradePosted(enrollmentId) THEN raise PolicyError('Review window closed')
4. IF stars < 1 OR stars > 5 THEN raise ValidationError
5. tabooWords ← DB.getTabooWords()  // cached list
6. count ← countTaboo(text, tabooWords)
7. IF count == 0
      published ← true; masked ← text
8. ELSE IF count IN [1, 2]
      masked ← maskTaboo(text, tabooWords)
      published ← true
      WarningService.issue(studentId, 'Taboo language in review (1-2 words)', count=1)
9. ELSE  // count >= 3
```

```
      published ← false; masked ← text
      WarningService.issue(studentId, 'Taboo language in review (3+ words)', count=2)
10. review ← DB.createReview({ enrollmentId, stars, text:masked, published,
tabooCount:count })
11. ClassRatingService.recompute(enrollment.sectionId)  // update avg rating
12. RETURN review
```

4.6 GradeService

submitGrades(instructorId, sectionId, grades{}) → void

```
INPUT:  instructorId (UUID), sectionId (UUID),
        grades: { enrollmentId → letterGrade }
OUTPUT: void OR PolicyError

1. section ← DB.getSection(sectionId)
2. IF section.instructorId != instructorId THEN raise ForbiddenError
3. IF getCurrentSemesterPhase() != 'grading' THEN raise PolicyError
4. roster ← DB.getEnrollments(sectionId, status='enrolled')
5. IF roster.map(e=>e.id) != grades.keys() THEN raise PolicyError('All students must be
graded')
6. FOR each {enrollmentId, letter} IN grades
      points ← gradeToPoints(letter)  // A=4.0 B=3.0 C=2.0 D=1.0 F=0.0
      DB.createGradeRecord({ enrollmentId, letterGrade:letter, gradePoints:points })
7. classAvg ← mean(grades.values().map(gradeToPoints))
8. IF classAvg > 3.5 OR classAvg < 2.5
      Notifier.alertRegistrar(instructorId, sectionId, classAvg)
9. GPAService.recompute(roster.map(e=>e.studentId))
```

4.7 GPAService

recompute(studentIds[]) → void

```
INPUT:  studentIds (UUID[])
OUTPUT: void (updates STUDENT_PROFILE records)

1. FOR each studentId IN studentIds
   grades ← DB.getAllGradeRecords(studentId)  // all semesters
   semGrades ← DB.getSemesterGradeRecords(studentId, currentSemesterId)
   overallGPA ← weightedMean(grades)   // sum(points*credits)/sum(credits)
   semesterGPA ← weightedMean(semGrades)
   DB.updateStudentProfile(studentId, { overallGPA, semesterGPA })
   applyGPARules(studentId, overallGPA, semesterGPA)
```

applyGPARules(studentId, overallGPA, semesterGPA) → void

```
INPUT:  studentId, overallGPA (decimal), semesterGPA (decimal)
OUTPUT: void (side effects: terminations, warnings, honors)

1. IF overallGPA < 2.0
   THEN DB.updateUser(studentId, {status:'terminated'}); RETURN
2. failedTwice ← DB.checkDoubleFailure(studentId)  // same course F twice
   IF failedTwice THEN DB.updateUser(studentId, {status:'terminated'}); RETURN
3. IF overallGPA BETWEEN 2.0 AND 2.25
   THEN WarningService.issue(studentId, 'Low GPA warning')
        Notifier.scheduleRegistrarInterview(studentId)
4. IF semesterGPA > 3.75 OR (semestersCompleted > 1 AND overallGPA > 3.5)
   THEN HonorService.awardHonor(studentId, currentSemesterId)
```

4.8 WarningService

issue(userId, reason, count=1) → void

```
INPUT:  userId (UUID), reason (string), count (int, default 1)
OUTPUT: void (side effects: suspension for student if threshold hit)

1. FOR i FROM 1 TO count
      DB.createWarningEvent({ userId, reason, issuedAt:now() })
      DB.incrementWarnings(userId)
2. user ← DB.getUser(userId)
3. IF user.role == 'student' AND user.warnings >= 3
      SuspensionService.suspend(userId, currentSemesterId)
4. IF user.role == 'instructor' AND user.warnings >= 3
      DB.updateUser(userId, { status: 'suspended' })
```

4.9 SuspensionService

suspend(studentId, semesterId) → void

```
INPUT:  studentId (UUID), semesterId (UUID)
OUTPUT: void

1. fineAmount ← DB.getRegistrarConfig('suspension_fine')
2. DB.createSuspensionEvent({ studentId, semesterId, fineAmount, issuedAt:now() })
3. DB.updateUser(studentId, { status:'suspended' })
4. enrollments ← DB.getActiveEnrollments(studentId, semesterId)
5. FOR each enrollment IN enrollments
      DB.updateEnrollment(enrollment.id, { status: 'dropped' })
6. DB.resetWarnings(studentId)  // warnings reset after suspension applied
```

4.10 HonorService

applyHonorToRemoveWarning(studentId, honorId) → void

```
INPUT:  studentId (UUID), honorId (UUID)
OUTPUT: void OR PolicyError

1. honor ← DB.getHonor(honorId)
2. IF honor.studentId != studentId THEN raise ForbiddenError
3. IF honor.consumed THEN raise PolicyError('Honor already used')
4. warnings ← DB.getWarnings(studentId, status='active')
5. IF warnings.length == 0 THEN raise PolicyError('No warnings to remove')
6. // Requires explicit student confirmation (enforced at controller level)
7. oldest ← warnings[0]  // remove earliest warning
8. DB.updateWarning(oldest.id, { removed: true })
9. DB.decrementWarnings(studentId)
10. DB.updateHonor(honorId, { consumed:true, consumedAt:now(),
warningRemovedId:oldest.id })
11. AuditLog.record(studentId, 'HONOR_USED', { honorId, warningId:oldest.id })
```

4.11 GraduationService

requestGraduation(studentId) → GraduationResult

```
INPUT:  studentId (UUID)
OUTPUT: { status: 'approved'|'rejected', reason: string }

1. profile ← DB.getStudentProfile(studentId)
2. completedClasses ← DB.getCompletedClasses(studentId, passingOnly=true)
3. IF completedClasses.length < 8
      RETURN { status:'rejected', reason:'Must complete 8 classes' }
4. requiredCourses ← DB.getRequiredCourses()  // registrar-configured list
5. completedCourseIds ← completedClasses.map(c => c.courseId)
6. missing ← requiredCourses.filter(r => r.courseId NOT IN completedCourseIds)
7. IF missing.length > 0
      WarningService.issue(studentId, 'Reckless graduation application')
      RETURN { status:'rejected', reason:'Missing required: '+missing.names }
8. DB.updateUser(studentId, { status:'graduated' })
9. RETURN { status:'approved', reason:'Bachelor degree conferred' }
```

4.12 AIAssistantService

query(userId, role, questionText) → AIResponse

```
INPUT:  userId (UUID), role (ENUM), questionText (string)
OUTPUT: { answer: string, grounded: bool, sources: string[], warning: bool }

1. filter ← buildScopeFilter(userId, role)
   // visitor → roleScope='public'
   // student → roleScope IN [public,student] AND owned classes
   // instructor → roleScope IN [public,student,instructor] AND own students
2. embedding ← EmbeddingAPI.embed(questionText)
3. chunks ← VectorDB.query(embedding, filter, topK=5)
4. maxScore ← max(chunks.map(c => c.score))
5. IF maxScore >= CONFIDENCE_THRESHOLD
   THEN context ← chunks.map(c=>c.text).join('\n')
        answer ← formatGroundedAnswer(questionText, context)
        RETURN { answer, grounded:true, sources:chunks.map(c=>c.docTitle), warning:false
}
6. ELSE
   THEN llmResp ← LLMApi.complete({ system: systemPrompt, user: questionText })
        RETURN { answer:llmResp, grounded:false, sources:[], warning:true }
```

5. System Screens & GUI Prototype

This section describes the major GUI screens by role and includes a prototype walkthrough for Course Registration (UC-05).

5.1 Screen Inventory

Screen	Role(s)	Key Elements
Public Landing Page	Visitor (all)	College intro, top/low-rated classes, top GPA students, Apply buttons
Student Application Form	Visitor	Name, email, prior GPA, personal statement, Submit
Instructor Application Form	Visitor	Name, email, department, credentials, Submit
Login Screen	All	Email, password, Login button; first-login redirect to reset
Password Reset Screen	All (first login)	New password, confirm, Save
Student Dashboard	Student	My enrollments, GPA, warnings, honors, tutorials for new students
Course Registration Screen	Student	Class list with filters, schedule view, conflict checker, Submit
My Reviews Screen	Student	List of submitted reviews with moderation status
Complaint Form	Student / Instructor	Target selection, description, requested action, Submit
Honor Management Screen	Student	Available honors, 'Use Honor' button with confirmation
Graduation Request Screen	Student	Eligibility summary, required-course checklist, Apply
Instructor Dashboard	Instructor	Assigned classes, student rosters, academic details
Grade Entry Screen	Instructor	Roster table with grade inputs, Submit, lock after submission
Waitlist Management Screen	Instructor	Ordered waitlist, Admit button per student
Registrar Dashboard	Registrar	Full system stats, pending applications, current phase, alerts
Application Review Screen	Registrar	Application details, Approve/Reject with justification field
Semester Control Screen	Registrar	Current phase, Advance Phase button, audit log
Course & Class Setup Screen	Registrar	Add/edit courses, class sections, assign instructors
Complaint Review Screen	Registrar	Complaint details, action dropdown, Resolve button
Required Courses Screen	Registrar	List of required courses, Add/Remove buttons
AI Assistant Screen	All (scoped)	Chat input, answer display, source citations, hallucination warning banner
AI Advisor Screen	Registrar / Instructor	At-risk student list, risk scores, suggested interventions

5. System Screens & GUI Prototype

This section describes the major GUI screens by role and includes a prototype walkthrough for Course Registration (UC-05).

5.1 Screen Inventory

Screen	Role(s)	Key Elements
Public Landing Page	Visitor (all)	College intro, top/low-rated classes, top GPA students, Apply buttons
Student Application Form	Visitor	Name, email, prior GPA, personal statement, Submit
Instructor Application Form	Visitor	Name, email, department, credentials, Submit
Login Screen	All	Email, password, Login button; first-login redirect to reset
Password Reset Screen	All (first login)	New password, confirm, Save
Student Dashboard	Student	My enrollments, GPA, warnings, honors, tutorials for new students
Course Registration Screen	Student	Class list with filters, schedule view, conflict checker, Submit
My Reviews Screen	Student	List of submitted reviews with moderation status
Complaint Form	Student / Instructor	Target selection, description, requested action, Submit
Honor Management Screen	Student	Available honors, 'Use Honor' button with confirmation
Graduation Request Screen	Student	Eligibility summary, required-course checklist, Apply
Instructor Dashboard	Instructor	Assigned classes, student rosters, academic details
Grade Entry Screen	Instructor	Roster table with grade inputs, Submit, lock after submission
Waitlist Management Screen	Instructor	Ordered waitlist, Admit button per student
Registrar Dashboard	Registrar	Full system stats, pending applications, current phase, alerts
Application Review Screen	Registrar	Application details, Approve/Reject with justification field
Semester Control Screen	Registrar	Current phase, Advance Phase button, audit log
Course & Class Setup Screen	Registrar	Add/edit courses, class sections, assign instructors
Complaint Review Screen	Registrar	Complaint details, action dropdown, Resolve button
Required Courses Screen	Registrar	List of required courses, Add/Remove buttons
AI Assistant Screen	All (scoped)	Chat input, answer display, source citations, hallucination warning banner
AI Advisor Screen	Registrar / Instructor	At-risk student list, risk scores, suggested interventions

5.2 Prototype: Course Registration (UC-05)

Below is a detailed screen prototype for the Course Registration flow, demonstrating the key UI states.

SCREEN 1: Available Classes

```
College0 — Course Registration            [Fall 2026]
Phase: Registration   Student: J. Smith  GPA: 3.2

[Search classes...         ] [Filter by: Department ▾]

□ CS101 – Intro to CS    MWF 9–10am   Cap: 30 (12 left)
□ CS201 – Data Structures TTh 11–12:30 Cap: 25 (0 left)
   → [Join Waitlist]
□ CS301 – Algorithms     MWF 2–3pm    Cap: 20 (5 left)
□ MATH101 – Calculus I   TTh 9–10:30  Cap: 35 (20 left)
□ ENG101 – Technical Writ MWF 11–12   Cap: 15 (3 left)

Selected: 0/4   [⚠️ Select 2–4 classes]
                                  [Submit Registration]
```

SCREEN 2: Conflict Error State

```
✓ CS101 – Intro to CS    MWF 9–10am   [selected]
✓ CS301 – Algorithms     MWF 2–3pm    [selected]
✓ MATH101 – Calculus I   TTh 9–10:30  [selected]
✓ ENG102 – Adv. Writing  TTh 9–10:30  [selected]

  ⛔ SCHEDULE CONFLICT DETECTED
  MATH101 and ENG102 overlap on TTh 9–10:30.
  Please remove one of these classes.

Selected: 4/4                 [Submit Registration ✗]
```

SCREEN 3: Registration Confirmation

```
          ✅ Registration Successful!

ENROLLED:
  • CS101 – Intro to CS        MWF 9–10am
  • CS301 – Algorithms         MWF 2–3pm
  • MATH101 – Calculus I       TTh 9–10:30

WAITLISTED:
  • CS201 – Data Structures    Position: #3
    (Instructor will notify if admitted)

Active courses: 3  (minimum met ✓)
                                [Go to My Dashboard]
```

6. Group Meeting Memos

Meeting 1 — March 19, 2026

Field	Details
Date	March 19, 2026, 6:00 PM
Attendees	Atai Kydyrov, Abdullah Zidan, Syed Mahmood, Justin Jones
Format	Discord voice call

Agenda & Discussion:

- Reviewed the full project specification document.
- Divided SRS sections among team members.
- Agreed on FastAPI (Python) backend, React/TypeScript frontend, PostgreSQL via Supabase.
- Identified highest-risk requirement areas: taboo moderation thresholds, semester phase transitions, GPA-driven automations.

Decisions:

- Atai: SRS structure and non-functional requirements. Abdullah: functional requirements and business rules. Syed: AI feature requirements. Justin: use-case reports and acceptance criteria.
- Target SRS submission: March 22.

Meeting 2 — March 21, 2026

Field	Details
Date	March 21, 2026, 7:30 PM
Attendees	Atai Kydyrov, Abdullah Zidan, Syed Mahmood, Justin Jones
Format	Discord voice call + shared Google Doc

Agenda & Discussion:

- Reviewed SRS draft together; identified missing edge cases in BR-13–BR-18.
- Discussed honor roll mechanics — confirmed honor-warning removal must be student-initiated.
- Clarified that suspension drops all active enrollments immediately.
- Finalized required-course definition for graduation check.

Decisions:

- Abdullah to add BR-13–BR-18 with detailed wording.
- Move technology stack to advisory Appendix A (not binding).
- SRS version 1.1 to be finalized March 23.

Meeting 3 — April 7, 2026

Field	Details
Date	April 7, 2026, 6:00 PM
Attendees	Atai Kydyrov, Abdullah Zidan, Syed Mahmood, Justin Jones
Format	Discord voice call

Agenda & Discussion:

- Kicked off Phase II design report planning.
- Identified feedback from grader: need proper use-case diagram with actors; FRs and UCs should be integrated.
- Agreed to restructure report around use-case diagram first, then per-UC scenarios.
- Discussed which 3 use cases to represent as Petri nets: UC-04 (semester phases), UC-07 (review moderation), UC-08 (grade submission). Chosen for their clear state-transition nature.

Decisions:

- Atai: E-R diagram and data model. Abdullah: pseudo-code for all service methods. Syed: AI service pseudo-code and system screens. Justin: Petri nets and sequence diagrams.

Meeting 4 — April 16, 2026

Field	Details
Date	April 16, 2026, 7:00 PM
Attendees	Atai Kydyrov, Abdullah Zidan, Syed Mahmood, Justin Jones
Format	Discord voice call + GitHub review session

Agenda & Discussion:

- Reviewed Phase II draft. Noted that diagrams lacked clear names and labels — aligned with grader feedback.
- Added explicit actor–UC diagram table and improved all diagram labels.
- Confirmed all FR IDs are referenced in use-case descriptions.
- Reviewed E-R diagram for completeness; added HONOR_EVENT, SUSPENSION_EVENT, REQUIRED_COURSE, and AI entities.
- Team concern: time pressure on implementation — need to prioritize core policy engine tests first.

Concerns & Mitigations:

- Risk: Business-rule complexity → Mitigation: dedicated unit tests per rule in Pytest.
- Risk: LLM API rate limits → Mitigation: cache frequent queries; graceful degradation to local retrieval.
- Risk: Integration conflicts close to deadline → Mitigation: feature branches per domain, weekly merge reviews.

7. Git Repository

All project materials — including this Design Report, the SRS v1.1, meeting notes, source code, and test suites — are maintained in the team's GitHub repository:

Repository URL	`https://github.com/team-y-csc322/college0`
Branch (main)	`main`
Branch (design docs)	`docs/phase-ii-design`
Branch (backend dev)	`feature/backend-core`
Branch (frontend dev)	`feature/frontend-spa`
Branch (AI subsystem)	`feature/ai-assistant`

Repository Structure:

```
college0/
├── docs/
│   ├── SRS-v1.1.pdf
│   ├── Phase-II-Design-Report.docx
│   └── meeting-notes/
├── backend/                  # FastAPI application
│   ├── app/
│   │   ├── auth/             # Authentication & RBAC
│   │   ├── academic/         # Semester, courses, enrollment
│   │   ├── conduct/          # Reviews, complaints, warnings
│   │   ├── ai/               # Vector retrieval & LLM fallback
│   │   └── analytics/        # Dashboard aggregations
│   ├── migrations/           # Alembic schema migrations
│   └── tests/                # Pytest unit & integration tests
├── frontend/                 # React + TypeScript SPA
│   ├── src/
│   │   ├── pages/
│   │   ├── components/
│   │   └── services/         # API clients
│   └── tests/                # Jest component tests
└── README.md
```